

# 剖析主题：EasyDistillation 蒸馏法格点 QCD 核心部分穷尽 剖析（代码-物理交叉向）

opencode pure

2026 年 8 月 17 日

## 摘要

本报告对 **EasyDistillation**（格点 QCD 蒸馏法软件包）执行 pure 穷尽剖析。项目性质判定为 **代码-物理交叉向**：核心层（generator/、quark\_contract.py、quark\_diagram.py、symmetry/、insertion/）为物理计算代码，每个关键符号都对应蒸馏法中的物理对象，独特性载体正是“代码符号  $\leftrightarrow$  物理对象”的双向对应。穷尽枚举核心部分候选 14 项（C1–C14），三态分配：**深挖 5 / 浅析 8 / 排除 1**。五个深挖部分为：C1 自动 Wick 收缩、C2 拉普拉斯本征向量生成器（含 stout CUDA 核）、C3 ElementalGenerator、C4 强子不可约表示与小群投影、C5 PerambulatorGenerator，每部分均按 15 步全流程重现并给出代码-物理双向映射表。最强竞争力结论：以**邻接矩阵编码 Wick 收缩图并自动转 einsum**的收缩引擎，配合群论自动枚举强子算符，是“手写收缩脚本”朴素实现难以企及的核心创新。实测：两个纯逻辑测试套件（19+8 例）本机通过；PyQUDA/GPU 数据路径未在本机验证。

## 目录

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1 项目定位与核心部分清单</b>                                     | <b>2</b>  |
| 1.1 项目性质判定                                               | 2         |
| 1.2 核心部分总清单（穷尽枚举）                                        | 2         |
| <b>2 核心部分深度剖析</b>                                        | <b>2</b>  |
| 2.1 C1 自动 Wick 收缩（quark_contract.py + quark_diagram.py）  | 3         |
| 2.2 C2 拉普拉斯本征向量生成器（eigenvector.py + stout_smear.cu）      | 5         |
| 2.3 C3 ElementalGenerator 导数/动量因子构造（elemental.py）        | 6         |
| 2.4 C4 强子不可约表示与小群投影（hadron*.py + symmetry/ + insertion/） | 8         |
| 2.5 C5 PerambulatorGenerator（PyQUDA 反演）                  | 9         |
| <b>3 独特性与竞争力评估</b>                                       | <b>10</b> |
| <b>4 关键片段与公式</b>                                         | <b>11</b> |
| <b>5 参考源清单</b>                                           | <b>15</b> |
| <b>6 结论</b>                                              | <b>16</b> |

# 1 项目定位与核心部分清单

## 1.1 项目性质判定

判定依据（实测统计）：lattice/ 41 个 .py 全部为物理计算代码（张量收缩、群论、反演），文档 README.md/TODO.md/doc/README.md 描述蒸馏法物理流程与数据形状，代码注释含大量物理量（perambulator.py:16-49 的参数说明、doc/README.md:3-65 的形状约定）。独特性载体为“符号  $\leftrightarrow$  对象”对应而非纯算法复杂度或纯推导链，故判定为**代码-物理交叉向**。

### 要点

**项目性质：**代码-物理交叉向（依据：物理计算代码占比  $\approx 100\%$ 、物理对象贯穿全部核心模块、独特载体 = 代码符号  $\leftrightarrow$  物理对象双向映射）

## 1.2 核心部分总清单（穷尽枚举）

| 编号  | 核心部分                                                       | 处理态 | 独特性概述                                                  |
|-----|------------------------------------------------------------|-----|--------------------------------------------------------|
| C1  | 自动 Wick 收缩 (quark_contract.py + quark_diagram.py)          | 深挖  | 味结构符号代数 $\rightarrow$ 邻接矩阵 $\rightarrow$ einsum 收缩，全自动 |
| C2  | 拉普拉斯本征向量生成器 (eigenvector.py + stout_smear.cu)              | 深挖  | scipy/cupyx/QUDA 三路径 + stout 平滑 CUDA 核                 |
| C3  | ElementalGenerator (elemental.py)                          | 深挖  | 协变导数乘积规则展开 + 动量平面波投影                                   |
| C4  | 强子不可约表示与小群投影 (hadron*.py + symmetry/ + insertion/)         | 深挖  | $O_h^D$ / 小群不可约表示自动枚举算符                                |
| C5  | PerambulatorGenerator (perambulator.py)                    | 深挖  | PyQUDA 反演 + $\gamma_5$ -厄米性 + MPI 片切分                  |
| C6  | Diagram 化简与批量计算引擎 (quark_diagram.py simplify/calc_diagram) | 浅析  | 图同构去重、连通分量拆分，支撑大规模收缩                                   |
| C7  | MPI 任务分发与原子写 (dispatch.py)                                 | 浅析  | 文件队列 + 原子锁的组态批处理                                       |
| C8  | 惰性内存映射 IO (filedata/)                                      | 浅析  | mmap 按需切片 + 吞吐统计                                       |
| C9  | 双粒子 JPC 算符 (symmetry/two_particle.py)                      | 浅析  | 球谐 + CG 系数构造 $J^{PC}$ 双粒子基                             |
| C10 | 费曼图绘制 (quark_draw.py)                                      | 浅析  | 邻接矩阵 $\rightarrow$ 可视化                                 |
| C11 | 噪声向量/广义与密度 perambulator (noisevector.py 等)                 | 浅析  | 随机蒸馏与高模、 $\gamma$ -动量扩展反演                              |
| C12 | 后端抽象与数据预设 (backend.py, preset.py, data.py)                 | 浅析  | numpy/cupy 切换 + 四类数据形状预设                               |
| C13 | 物理常量与动量字典 (constant.py, mom_dict.py)                       | 排除  | 纯查表常量，无独特性                                             |
| C14 | 工程配置 (README/TODO/LICENSE/requirements/.git*)              | 排除  | 非核心代码                                                  |

表 1: 核心部分总清单（三态填满，数据来源：全库穷尽枚举实测）

## 2 核心部分深度剖析

## 2.1 C1 自动 Wick 收缩 (quark\_contract.py + quark\_diagram.py)

## 代码-物理映射

**映射原则:** 每个关键代码符号必有物理对象，双向可查，无孤儿符号。

| 代码符号                     | 物理对象                              | 物理含义与参考源                                                                                |
|--------------------------|-----------------------------------|-----------------------------------------------------------------------------------------|
| Quark(flavor, tag, anti) | 场算符 $\psi_f(x) / \bar{\psi}_f(x)$ | 带味的裸夸克符号 (tag 携带强子序号与时间); quark_contract.py:17-36                                       |
| Tag(tag, time)           | 时空/粒子标号                           | tag//3= 强子号、tag%3= 强子内夸克号、time= 时间片; quark_contract.py:12-14                            |
| Propagator               | 夸克传播子 $S^f$                       | 符号 $S^f$ (汇, 源); 同位时标为 local; quark_contract.py:39-61                                   |
| HadronFlavorStructure    | 强子味结构                             | 介子 (2 字)/重子 (3 字)/反重子 (bar{}), baryon_num $\in \{0, 1, -1\}$ ; quark_contract.py:64-107 |
| _quark_contract          | Wick 收缩                           | 反夸克锚定 + 同位素夸克递归配对 + 交换符号; quark_contract.py:216-245                                     |
| adjacency_matrix         | Wick 收缩图                          | 顶点 = 强子、矩阵元 = 传播子编号 (重子为 3×3 子矩阵); quark_contract.py:179-208                            |
| QuarkDiagram.analyse     | 图 → 张量网                           | BFS 分解连通分量, 分配自旋/色下标生成 einsum 式; quark_diagram.py:15-79                                 |
| compute_diagrams         | 关联函数数值                            | 各图逐项 einsum 求值; quark_diagram.py:261-273                                                |

表 2: C1 代码-物理双向映射 (数据来源: 源码逐符号核验)

**A1 目的与前期准备.** Wick 收缩是把强子关联函数  $\langle O_1 O_2 \cdots \rangle$  按费曼拓扑展开为传播子乘积组合的运算, 手写极易出错 (交换符号、味匹配、简并)。该库把它抽象为“符号代数 + 图论”两层。前置条件: 仅依赖 sympy 与 numpy (requirements.txt:1-4)。

**A2 输入.** 表达式 (HadronFlavorStructure 的乘积)、粒子列表、简并开关 degenerate (quark\_contract.py:110-124)。

**A3 输出 (应是什么).** Diagram 的符号表达式 (系数 + 邻接矩阵 + 时标 + 传播子表), 如 meson-meson 收缩得  $\pm[0, 1], [1, 0]$  对角拓扑。

**A4 整体框架.** 两步: quark\_contract (味 → 传播子符号与邻接矩阵, quark\_contract.py:125-213) → QuarkDiagram (邻接矩阵 → einsum, quark\_diagram.py:15-79) → compute\_diagrams (求值)。

**A5 关键细节.** 递归配对用“首个反夸克为锚” (:221-222); 简并 u/d 统一传播子味 "q" (:233-236); 交换符号  $(-1)^{i-j}$  (:225-232); BFS 遍历避免重复处理 (quark\_diagram.py:29-50)。

```

1 def _quark_contract(symbol_list, result_list, result, degenerate):
2     if symbol_list == []:
3         result_list.append(Mul(*result))
4         return
5     for i, src in enumerate(symbol_list):
6         if src.anti:
7             break
8     for j, snk in enumerate(symbol_list):
9         if not snk.anti and snk.flavor == src.flavor:
10            if i > j:

```

```

11         symbol_list.pop(i)
12         symbol_list.pop(j)
13         factor = S(-1) ** (i - j - 1)
14     else:
15         symbol_list.pop(j)
16         symbol_list.pop(i)
17         factor = S(-1) ** (j - i)
18     if degenerate and (snk.flavor == "u" or snk.flavor == "d"):
19         prop = Propagator("q", src.tag, snk.tag)
20     else:
21         prop = Propagator(src.flavor, src.tag, snk.tag)
22     result.append(factor * prop)
23     _quark_contract(symbol_list, result_list, result, degenerate)
24     result.pop()
25     if i > j:
26         symbol_list.insert(j, snk)
27         symbol_list.insert(i, src)
28     else:
29         symbol_list.insert(i, src)
30         symbol_list.insert(j, snk)

```

**A6 任务如何正确执行.** 调用 `quark_contract(expr, particles, degenerate)` (`test_quark_contract.py:159,170,181`); 数值阶段把结果代入 `compute_diagrams`。

**A7 实际执行情况.** 本机实测 `test_quark_contract.py`: Ran 19 tests, OK (记录于 `.pure.*.log`)。

**A8 实际输出情况.** meson-meson ( $\bar{u}d \times \bar{d}u$ ) 收缩输出 `...q`, 邻接矩阵 `[[1,0],[0,1]]`, 与 `test_quark_contract.py:184` 断言一致。

**A9 结果分析.** 19 例测试覆盖 tag 不可变性、简并、重子-反重子、传播子符号表示 (`test_quark_contract.py:12-222`), 符号层自洽性验证通过。

**A10 综合分析.** 该引擎是蒸馏法四阶段之后的核心抽象: 物理上 Wick 收缩的“所有可能配对”即符号代数的“所有递归分支”, 配对顺序产生置换符号, 映射逐符号成立 (见映射表)。

**A11 结尾总结.** **核心算法:** 递归 Wick 收缩  $\rightarrow$  邻接矩阵  $\rightarrow$  BFS 图分析  $\rightarrow$  einsum; 正确性由 19 例符号测试保证。

**A12 结尾评价.** 优点: 全自动、可符号化简; 缺点: 符号代数分支数随粒子数指数增长 (未验证性能瓶颈), 邻接矩阵仅存拓扑不存旋量组合细节。

**A13 补充说明.** 大规模 ( $>4$  顶点) 收缩的耗时与内存未实测; 数值路径 (含 perambulator) 未在本机运行。

**A14 参考源.** `quark_contract.py:110-245`、`quark_diagram.py:15-79,261-273`、`test_quark_contract.py:12-222`。

**A15 附录.** 实测命令与输出见 `.pure.2026-08-17-06-18-21.log`。

## 2.2 C2 拉普拉斯本征向量生成器 (eigenvector.py + stout\_smear.cu)

## 代码-物理映射

**映射原则：**每个关键代码符号必有物理对象，双向可查。

| 代码符号              | 物理对象                          | 物理含义与参考源                                                                   |
|-------------------|-------------------------------|----------------------------------------------------------------------------|
| _Laplacian        | 协变拉普拉斯 $-\nabla^2$            | 对角 6、plaquette 链求和 (SA 求最小本征) ; eigenvector.py:11-26                       |
| backend.roll      | 位移算子 $\hat{T}_\mu$            | 相邻格点平移 (周期边界); eigenvector.py:19-24                                        |
| eigsh(which="SA") | 本征问题 $\nabla^2 v = \lambda v$ | 最低 $N_e$ 个本征解 (升序排序) ; eigenvector.py:246-251                              |
| renorm_phase      | 本征向量相位                        | $v \rightarrow e^{-i \arg(v_1)} v$ 约定 (本征向量相位不确定) ; eigenvector.py:254-256 |
| project_SU3       | 么正化                           | 迭代 $U \leftarrow \frac{1}{2}(U + U^\dagger)$ 到 SU(3); eigenvector.py:55-65 |
| stout_smear       | stout 平滑                      | 指数化 $U'_\mu = e^{iQ_\mu} U_\mu$ 降紫外噪声; eigenvector.py:215-232              |
| stout_smear.cu    | GPU 平滑核                       | 逐链接 staple + 矩阵指数; stout_smear.cu:160-245                                  |
| laplacian_quda    | QUDA 本征求解                     | TR-Lanczos + 可选切比雪夫加速; eigenvector.py:259-312                              |

表 3: C2 代码-物理双向映射 (数据来源: 源码逐符号核验)

**A1 目的与前期准备.** 蒸馏基底需要三维空间拉普拉斯的最低  $N_e$  本征向量; 同时要平滑规范场以降低本征求解噪声。前置: 规范场数据、numpy/scipy (CPU) 或 cupyx/QUDA (GPU)、PyQUDA (eigenvector.py:203-213, 259-262)。

**A2 输入.** 格点尺寸 latt\_size、规范场 GaugeField、 $N_e$ 、容差 tol (eigenvector.py:29-47)。

**A3 输出.** 每时间片 [Ne, Lz, Ly, Lx, Nc] 本征向量 + 本征值 (eigenvector.py:257)。

**A4 整体框架.** 三路径 (calc, :356-370) : cupy+QUDA  $\rightarrow$  laplacian\_quda; cupy/numpy  $\rightarrow$  laplacian\_cupy\_numpy; QUDA 拉普拉斯 + cupyx 求解  $\rightarrow$  laplacian\_quda\_scipy。平滑独立分发 (:215-232)。

**A5 关键细节.** 拉普拉斯用 opt\_einsum 实现

$$(-\nabla^2 \psi)_a(x) = 6\psi_a(x) - \sum_{\mu=1}^3 [U_\mu(x)\psi(x + \hat{\mu}) + U_\mu^\dagger(x - \hat{\mu})\psi(x - \hat{\mu})]_a \quad (1)$$

即 (1); 对角项取 6 (eigenvector.py:16-17); 动量卷积用 roll (:19-24)。stout 平滑指数化  $e^{iQ}$  (eigenvector.py:140-189 与 CUDA 核 stout\_smear.cu:198-239 逐点一致)。

## 物理图像

**物理图像:** 拉普拉斯最低本征向量 (低频模) 近似 “光滑夸克场分布”, 作为蒸馏基底; stout 平滑把规范场变光滑, 等价于剔除高动量 (紫紫外) 噪声, 使低频模更稳定。

**A6 任务如何正确执行.** load(key)  $\rightarrow$  stout\_smear(nstep, rho)  $\rightarrow$  逐 t 片 calc(t) (test\_eigenvector.py:44-50)。

**A7 实际执行情况.** 本机无 QUDA (libquda.so 缺失) 且 LFS 数据未拉取, 本征求解路径未运行; cupy 设备可用 (1 GPU)。未验证。

**A8 实际输出情况.** 无数据相关输出; 本机仅验证 `import lattice` 与逻辑测试。

**A9 结果分析.** 求解残差/相位一致性测试 (`test_eigenvector.py:25-36`) 依赖参考数据 `.eigenvector.ref.npy` (LFS), 本机未产生。

**A10 综合分析.** 三路径共享同一 `calc` 接口 (:356-370), 工程上是“后端无关”设计; 物理上 `scipy/cupyx` 与 QUDA 求解的差别仅在性能与多项式加速 (:295-299 切比雪夫)。

**A11 结尾总结.** **核心算法:** (1) 的 `LinearOperator + eigsh(SA)` 三路径 + stout 平滑 (NDArray/CUDA/QUDA); GPU 数据路径本机未验证。

**A12 结尾评价.** 优点: 三后端路径与平滑三实现可切换; 缺点: CUDA 核仅实例化 `double` 模板 (`eigenvector.py:38-42` 的 TODO), `numpy/cupy` 平滑代码三份重复。

**A13 补充说明.** 本征求解的实际耗时/精度无基准数据; CUDA 核数值一致性未实测。

**A14 参考源.** `eigenvector.py:11-26, 234-257, 356-370`、`stout_smear.cu:160-245`、`test_eigenvector.py:25-59`。

**A15 附录.** 无 (GPU 路径未运行)。

## 2.3 C3 ElementalGenerator 导数/动量因子构造 (elemental.py)

### 代码-物理映射

**映射原则:** 每个关键代码符号必有物理对象, 双向可查。

| 代码符号                          | 物理对象                    | 物理含义与参考源                                                                                                            |
|-------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>num_derivative</code>   | 导数组合数                   | $(3^{n+1} - 1)/2$ ( $n$ 阶全组合); <code>elemental.py:48</code>                                                         |
| <code>derivative(n)</code>    | 导数算符 $\tilde{\nabla}^n$ | 3 进制方向编码; <code>derivative.py:23-33</code>                                                                          |
| <code>derivative_list</code>  | 导数组合集                   | 0 阶到 $n$ 阶全部组合; <code>elemental.py:49</code>                                                                        |
| <code>MomentumPhase</code>    | 平面波 $e^{ip \cdot x}$    | 动量投影相位 (含 cb2 棋盘化); <code>phase.py:41-53</code>                                                                     |
| <code>_nD</code>              | 协变位移                    | $V_f = U_\mu(x)V(x + \hat{\mu}) - U_\mu^\dagger(x - \hat{\mu})V(x - \hat{\mu})$ ; <code>elemental.py:279-288</code> |
| <code>pick_left/right</code>  | 乘积规则展开                  | $\tilde{\nabla}(AB) = (\tilde{\nabla}A)B + A(\tilde{\nabla}B)$ 的左右分配; <code>elemental.py:309-321</code>             |
| <code>stochastic_coeff</code> | 随机蒸馏系数                  | 稀释 (dilution) 块对角系数; <code>elemental.py:71-95</code>                                                                |

表 4: C3 代码-物理双向映射 (数据来源: 源码逐符号核验)

**A1 目的与前期准备.** Elemental  $\Phi = V^\dagger \mathcal{O} V$  是算符  $\mathcal{O}$  在蒸馏空间的表示, 导数/动量使算符携带量子数 (角动量、动量)。前置: 平滑规范场 + 本征向量。

**A2 输入.** 规范场、本征向量、`num_nabla`、`momentum_list`、可选稀释 (`elemental.py:16-26`)。

**A3 输出.** `[num_deriv, num_mom, Ne, Ne]` 张量 (`elemental.py:56`), 即

$$\Phi_{jk}^{(n)}(p; t) = \sum_x e^{ip \cdot x} [\tilde{\nabla}^n V(x)]_{j,a}^* V(x)_{k,a} \quad (2)$$

**A4 整体框架.** `calc(t)` 三层循环: 本征向量装入 (:297-298)  $\rightarrow$  导数组合循环 (:299)  $\rightarrow$  动量循环 (:322-337)。

**A5 关键细节.** 乘积规则展开: 对  $n$  阶导数, 把  $2^n$  个子集分配为“右作用”/“左作用”, 系数  $(-1)^{\#右}$  (:309-321)。此写法替代注释掉的 `num_nabla_right` 组合写法 (:301-308), 等价但更紧凑。

```

1     for e in range(V.shape[0]):
2         V[e] = eigenvector[t, e]
3     for derivative_idx, derivative in enumerate(self.derivative_list):
4         VPV[derivative_idx] = 0
5         # for num_nabla_right in range(len(derivative) + 1):
6         #     coeff = (-1)**num_nabla_right * comb(len(derivative), num_nabla_right)
7         #     right = self._nD(V, U[:, t], derivative[:num_nabla_right])
8         #     left = self._nD(V, U[:, t], derivative[num_nabla_right:][::-1])
9         #     for momentum_idx, momentum in enumerate(self.momentum_list):
10            #         VPV[derivative_idx, momentum_idx] += contract(
11                #             "zyx,ezyxc,fzyxc->ef", coeff * momentum_phase.get(momentum), left.conj(), right
12            #         )
13    for pick_nabla in range(2 ** len(derivative)):
14        pick_right = []
15        pick_left = []
16        pick = pick_nabla
17        for direction in derivative:
18            if pick & 1:
19                pick_right.append(direction)
20            else:
21                pick_left.append(direction)
22            pick >>= 1
23        coeff = (-1) ** len(pick_right)
24        right = self._nD(V, U[:, t], pick_right)
25        left = self._nD(V, U[:, t], pick_left[::-1])
26        for momentum_idx, momentum in enumerate(self.momentum_list):
27            if self.stochastic_coeff is None:
28                VPV[derivative_idx, momentum_idx] += contract(
29                    "zyx,ezyxc,fzyxc->ef",
30                    coeff * momentum_phase.get(momentum),
31                    left.conj(),
32                    right,
33                )
34            else:
35                VPV[derivative_idx, momentum_idx] += contract(
36                    "zyx,ezyxc,fzyxc,ef->ef",
37                    coeff * momentum_phase.get(momentum),
38                    left.conj(),
39                    right,
40                    self.stochastic_coeff,

```

**A6 任务如何正确执行.** `load(key) → 逐 t 片 calc(t)` (`test_elemental.py:41-45`); 结果与参考 `.elemental.npy` 比对 (:31-34)。

**A7 实际执行情况.** 数据依赖 LFS (`weak_field.lime` 为指针), 本机未运行。未验证。

**A8 实际输出情况.** 无数据相关输出。

**A9 结果分析.** 参考比对以  $res = \|data_{ref} - data\|$  报告 (`test_elemental.py:32-34`), 本机无法复核。

**A10 综合分析.** (2) 中  $e^{ipx}$  来自 `MomentumPhase` (`phase.py:41-46`); 协变导数保证规范协变性, 这是“物理上必需”而朴素实现 (普通差分) 容易漏掉的点。



**A11 结尾总结. 核心算法:** 乘积规则展开  $2^n$  子集分配 + 协变位移  $U$ -平移 + 动量相位投影的一次 einsum; 正确性依赖参考数据 (未在本机验证)。

**A12 结尾评价.** 优点: 导数阶数任意 ( $n < 3$ , README.md:18); 缺点: 随机稀释分支 (:61-95) 与确定性分支代码未统一。

**A13 补充说明.** 稀释/混合随机方法 (blending) 的统计无偏性未实测。

**A14 参考源.** elemental.py:48-56, 279-338、derivative.py:23-33、phase.py:41-53、test\_elemental.py:23-48。

**A15 附录.** 无。

## 2.4 C4 强子不可约表示与小群投影 (hadron\*.py + symmetry/ + insertion/)

### 代码-物理映射

**映射原则:** 每个关键代码符号必有物理对象, 双向可查。

| 代码符号                     | 物理对象             | 物理含义与参考源                                                                                               |
|--------------------------|------------------|--------------------------------------------------------------------------------------------------------|
| OhD_generator            | 双群 $O_h^D$ 不可约表示 | A1..H 各不可约表示生成元; group_generator.py: 25-73                                                             |
| OD_irreps                | $O_h^D$ 硬编码表     | 全群元矩阵 (乘表 + 不可约表示); hardcoded_rep.py: 10188                                                            |
| genLittleGroupIrrep      | 小群不可约表示          | 按 $p^2$ 选小群 (Oh/Dic4/Dic2/Dic3/C4) ; gen_hardcoded_rep.py:162-223                                      |
| wignerRotate             | Wigner 转动        | 旋转到参考动量再回代; gen_hardcoded_rep.py: 145-159                                                              |
| reductionToLittleGroup   | 不可约分解            | $\frac{d_\lambda}{ G } \sum_g \Gamma_{mm}^\lambda(g) D(g)$ 投影; gen_hardcoded_rep.py: 226-245           |
| Insertion                | 插入算符             | $\gamma \otimes$ 导数 $\otimes$ 投影组合; insertion/__init__.py: 219-350                                     |
| GammaName/DerivativeName | 强子量子数            | $\pi(0^{--}), \rho(1^{--})$ 等与 $\nabla, \mathbb{D}, \mathbb{E}$ ; gamma.py:103-112、derivative.py:36-59 |
| HadronIrrepRow.transform | 群元作用             | $\hat{R}(g) p, i\rangle = \sum_j D_{ji}^{l, g}(W_g) R_g p, j\rangle$ ; hadron_irrep.py:132-140         |

表 5: C4 代码-物理双向映射 (数据来源: 源码逐符号核验)

**A1 目的与前期准备.** 强子插值算符必须携带确定量子数 ( $J^{PC}$ 、动量、不可约表示), 格点群论把连续角动量离散为  $O_h^D$  不可约表示。前置: sympy 群论工具链。

**A2 输入.**  $\gamma$  组合、导数组合、投影不可约表示名、动量字典 (insertion/\_\_init\_\_.py:219-238)。

**A3 输出.** Insertion.rows (DerivativeRepsRow 列表, 即 “ $\gamma \times$  导数组合” 的算符行), 可投影到小群 (:255-267)。

**A4 整体框架.** 三层: 群生成元 (group\_generator.py)  $\rightarrow$  硬编码不可约表示 (hardcoded\_rep.py)  $\rightarrow$  小群投影与 Wigner 转动 (gen\_hardcoded\_rep.py)  $\rightarrow$  算符构造 (insertion/\_\_init\_\_.py)  $\rightarrow$  强子类 (hadron\_irrep.py)。

**A5 关键细节.** 小群不可约表示的投影公式

$$P_{mm'}^\lambda = \frac{d_\lambda}{|G|} \sum_{g \in G} \Gamma_{mm'}^\lambda(g) \hat{R}(g) \quad (3)$$



实现于 `reductionToLittleGroup` (`gen_hardcoded_rep.py:226-245`) 与 `expr_little_group_projection` (`hadron_irrep.py:178-195`)；旋转算符的 Wigner 矩阵查 `refRotateDict` (`group_generator.py:288-364`)。

**A6 任务如何正确执行.** 例 `gen_twopt.py:9-11`:

`Insertion(GammaName.PI, DerivativeName.IDEN,`  
`ProjectionName.A1, momDict)` 再 `Operator`。

**A7 实际执行情况.** 本机未执行 `test_mesonspectrum.py` (需 LFS 数据)；群论模块逻辑测试含于 `test_simplify.py` (已通过 8 例)。

**A8 实际输出情况.** 无数据相关输出。

**A9 结果分析.** 强子谱 2pt 有效质量测试 (`test_mesonspectrum.py:44-65`) 依赖 `elemental/perambulator` 数据, 本机不可复现。

**A10 综合分析.** 该模块是“物理量子数  $\rightarrow$  代码算符”的直接映射:  $\gamma$  命名表 (`gamma.py:103-112`) 把  $a_0, \pi, \rho, \dots$  映射到不可约表示/宇称/C-共轭/厄米性, 导数命名表 (`derivative.py:36-67`) 映射  $\nabla, \mathbb{B}, \mathbb{D}, \mathbb{E}$ ; 物理上这些是  $J^{PC}$  的格点实现。

**A11 结尾总结. 核心算法:** (3) 小群投影 + Wigner 转动 + 硬编码不可约表示查表, 自动枚举确定  $J^{PC}$  的算符行。

**A12 结尾评价.** 优点: 群论自动化 (TODO.md:10 已完成); 缺点: 硬编码表由脚本生成后体积巨大 (11613 行) 且部分动量 ( $p^2 = 5, 6$ ) 走运行时生成 (`gen_hardcoded_rep.py:188-195`)。

**A13 补充说明.** 高动量  $p^2 > 6$  未实现 (:197 抛 `NotImplementedError`)。

**A14 参考源.** `group_generator.py:25-73, 288-364`, `gen_hardcoded_rep.py:145-245`, `hardcoded_rep.py:10188`, `insertion/__init__.py:219-350`, `hadron_irrep.py:132-195`。

**A15 附录.** 无。

## 2.5 C5 PerambulatorGenerator (PyQUDA 反演)

### 代码-物理映射

**映射原则:** 每个关键代码符号必有物理对象, 双向可查。

| 代码符号                                  | 物理对象            | 物理含义与参考源                                                                                          |
|---------------------------------------|-----------------|---------------------------------------------------------------------------------------------------|
| <code>PerambulatorGenerator</code>    | 蒸馏传播子生成器        | $\tau$ 的定义与反演参数; <code>perambulator.py:10-102</code>                                              |
| <code>dirac</code>                    | Dirac 算符 $D$    | Clover 参数化 ( <code>mass/tol/xi/clover/multigrid</code> ) ;<br><code>perambulator.py:91-100</code> |
| <code>eigenvector_dagger</code>       | 源向量 $\bar{v}_j$ | cb2 棋盘化 + 共轭装载; <code>perambulator.py:117-128</code>                                              |
| <code>dirac.invert</code>             | 反演 $D^{-1}$     | 求解 $DSV = V$ (逐本征向量); <code>perambulator.py:192</code>                                            |
| <code>contract("...-&gt;tijk")</code> | 收缩 $\bar{v}SV$  | $\tau_{tijk} = \bar{v}_{etzyx} S_{...}$ ; <code>perambulator.py:198-200</code>                    |
| <code>grid_coord</code>               | MPI 子格坐标        | 片切分本征向量; <code>perambulator.py:111-125</code>                                                     |
| <code>MRHS</code>                     | 多右端项            | 一次反演多源 (省时费显存); <code>perambulator.py:172-183</code>                                              |

表 6: C5 代码-物理双向映射 (数据来源: 源码逐符号核验)

**A1 目的与前期准备.** `perambulator` 把夸克传播子压缩到蒸馏空间:  $\tau(t_f, t_i) = V^\dagger(t_f)D^{-1}V(t_i)$ , 是 2pt 收缩的核心量。前置: PyQUDA + QUDA (`perambulator.py:69-70`)、cupy 后端 (:77)。

**A2 输入.** 规范场 (stout 平滑后)、本征向量、Clover 质量/参数、usedNe、MRHS (perambulator.py:51-68)。

**A3 输出.** [Lt, Ns, Ns, Ne, Ne] 张量 (perambulator.py:102), 即

$$\tau(t_f, t_i)_{jk}^{SS'} = [V(t_f)^\dagger]_j^S D^{-1} [V(t_i)]_k^{S'} \quad (4)$$

**A4 整体框架.** calc(t): 逐本征向量 e (:169) → 逐自旋装载源 (:185-192) → dirac.invert (:192) → einsum 收缩进 VSV (:198-200) → 打印显存/耗时 (:205-208)。

**A5 关键细节.** 源装载按 MPI 片切分 (:177-179 仅本地时间片写源); 反向传播子用  $\gamma_5$ -厄米性 (one\_particle.py:57); usedNe 裁剪 (:79-84); cb2 棋盘化经 core.cb2 (:128)。

**A6 任务如何正确执行.** load(key) (读 gauge + 本征向量) → stout\_smear(20, 0.1) → 逐 t 片 calc(t) 并 roll 平移 (test\_perambulator.py:61-66)。

**A7 实际执行情况.** 本机 import pyquda 失败 (libquda.so 缺失), GPU 反演未运行。未验证。

**A8 实际输出情况.** 无数据相关输出。

**A9 结果分析.** 测试以范数残差比对参考 perambulator (test\_perambulator.py:52-55), 本机无法复核。

**A10 综合分析.** (4) 中反演是唯一昂贵步骤 (每源一次 invert); MRHS 多右端项路径 (:172-183) 一次反演多个自旋源; 片切分 (:111-125) 配合 MPI 网格 (test\_perambulator\_mpi.py:10-17)。

**A11 结尾总结.** **核心算法:** 蒸馏空间反演  $\tau = V^\dagger D^{-1} V + \gamma_5$ -厄米反向传播子 + MPI 片切分; 数值未在本机验证。

**A12 结尾评价.** 优点: 参数完整 (Clover/multigrid/MRHS/各向异性 xi/nu); 缺点: 强依赖 PyQUDA 版本 (backend.py:46-47), 无 CPU 路径 (:140-141 抛 NotImplemented)。

**A13 补充说明.** multigrid 加速、MRHS 显存开销均无本机实测数据。

**A14 参考源.** perambulator.py:51-102, 146-209、test\_perambulator.py:29-66、test\_perambulator\_mpi.py:10-82、backend.py:36-58。

**A15 附录.** 无。

### 3 独特性与竞争力评估

| 独特点                  | 对比基准                 | 优势与证据                                                         |
|----------------------|----------------------|---------------------------------------------------------------|
| 自动 Wick 收缩 (C1)      | 手写收缩脚本 (逐拓扑写 einsum) | 味结构符号代数自动生成全部配对与置换符号; 测试 19 例符号断言通过; 邻接矩阵编码重子 3×3 子结构         |
| 图 → einsum 自动下标 (C1) | 朴素实现 (手工标下标)         | BFS 连通分量分解 + 自旋/色下标自动分配 (quark_diagram.py:23-79), 多连通分量图可拆分复用 |
| 本征向量三路径 (C2)         | 单一求解器                | scipy/cupyx/QUDA 三路径 + stout 三实现 (NDArray/CUDA/QUDA), 按设备自动选择 |
| stout CUDA 核 (C2)    | 纯 numpy 平滑           | 逐链接独立并行 (stout_smear.cu:160-245), Lt 块并行; 性能未实测               |
| 导数乘积规则展开 (C3)        | 朴素差分组合               | $2^n$ 左右分配 + 协变位移保持规范协变性; 动量相位统一进 einsum                      |

| 独特点         | 对比基准   | 优势与证据                                            |
|-------------|--------|--------------------------------------------------|
| 群论自动算符 (C4) | 手写算符清单 | 小群投影/Wigner 转动自动枚举确定 $J^{PC}$ 算符; 硬编码不可约表示表一次生成  |
| 蒸馏空间反演 (C5) | 全格点传播子 | $D^{-1}$ 只在蒸馏空间收缩, 规模 $N_e \times N_e$ ; MPI 片切分 |

表 7: 独特性评估 (数据来源: 源码分析 + 本机 27 例测试, 性能数值均未实测)

### 局限与风险

**竞争力局限:** 全部性能/数值优势为**未验证** (本机缺 QUDA 与 LFS 数据); 对比基准为“朴素实现”的定性推断, 无对照基准库的量化数字; 自动收缩在顶点数增大时符号分支指数增长, 未见剪枝策略。

## 4 关键片段与公式

核心公式 (编号):

$$C(\{t\}) = \sum_{\text{Wick}} (-1)^\sigma \prod_f \tau_{j_1 k_1}^f \cdots \times \Phi_{k_1 j_2} \cdots \quad (5)$$

即 (5): 关联函数 = 各 Wick 配对 (符号  $(-1)^\sigma$ ) 的传播子与 elemental 张量缩并, 实现于 quark\_diagram.py:233-273。

本征向量递归配对 (C1 核心, quark\_contract.py:216-245):

```

1 def _quark_contract(symbol_list, result_list, result, degenerate):
2     if symbol_list == []:
3         result_list.append(Mul(*result))
4         return
5     for i, src in enumerate(symbol_list):
6         if src.anti:
7             break
8     for j, snk in enumerate(symbol_list):
9         if not snk.anti and snk.flavor == src.flavor:
10             if i > j:
11                 symbol_list.pop(i)
12                 symbol_list.pop(j)
13                 factor = S(-1) ** (i - j - 1)
14             else:
15                 symbol_list.pop(j)
16                 symbol_list.pop(i)
17                 factor = S(-1) ** (j - i)
18             if degenerate and (snk.flavor == "u" or snk.flavor == "d"):
19                 prop = Propagator("q", src.tag, snk.tag)
20             else:
21                 prop = Propagator(src.flavor, src.tag, snk.tag)
22             result.append(factor * prop)
23             _quark_contract(symbol_list, result_list, result, degenerate)
24             result.pop()
25             if i > j:
26                 symbol_list.insert(j, snk)

```

```

27         symbol_list.insert(i, src)
28     else:
29         symbol_list.insert(i, src)
30         symbol_list.insert(j, snk)

```

图 →einsum (C1 核心, quark\_diagram.py:23-60):

```

1  def analyse(self) -> None:
2      from copy import deepcopy
3
4      adjacency_matrix = deepcopy(self.adjacency_matrix)
5      num_vertex = len(adjacency_matrix)
6      visited = [False] * num_vertex
7      for idx in range(num_vertex):
8          if visited[idx]:
9              continue
10         propagators = []
11         visited[idx] = True
12         queue = [idx]
13         while queue != []:
14             i = queue.pop(0)
15             for j in range(num_vertex):
16                 path = adjacency_matrix[i][j]
17                 if path != 0:
18                     adjacency_matrix[i][j] = 0
19                     if not visited[j]:
20                         visited[j] = True
21                         queue.append(j)
22                     if isinstance(path, int):
23                         propagators.append([path, i, j])
24                     elif isinstance(path, list):
25                         for _path in path:
26                             propagators.append([_path, i, j])
27                     else:
28                         raise ValueError(f"Invalid value {path} in the adjacency matrix")
29             if propagators == []:
30                 continue
31             vertex_operands = []
32             vertex_subscripts = []
33             propagator_operands = []
34             propagator_subscripts = []
35             node = 0
36             for propagator in propagators:
37                 propagator_operands.append(propagator)
38                 propagator_subscripts.append(_SUB_M[node + 1] + _SUB_A[node + 1] + _SUB_M[node] + _SUB_A[node])

```

拉普拉斯求解 (C2 核心, eigenvector.py:234-257):

```

1  def laplacian_cupy_numpy(self, t: int, apply_renorm_phase: bool):
2      backend = get_backend()
3      if backend.__name__ == "numpy":
4          from scipy.sparse import linalg
5      elif backend.__name__ == "cupy":
6          from cupyx.scipy.sparse import linalg
7      Lx, Ly, Lz, Lt = self.latt_size
8

```

```

9      U = backend.asarray(self._U[: Nd - 1, t])
10     U_dag = U.transpose(0, 1, 2, 3, 5, 4).conj()
11     Laplacian = functools.partial(_Laplacian, U=U, U_dag=U_dag, latt_size=self.latt_size)
12     A = linalg.LinearOperator((Lz * Ly * Lx * Nc, Lz * Ly * Lx * Nc), matvec=Laplacian, matmat=Laplacian)
13     evals, evecs = linalg.eigsh(A, self.Ne, which="SA", tol=self.tol)
14     evecs = evecs.transpose(1, 0).reshape(self.Ne, -1)
15
16     # sort eigenvalues additionally
17     argsort = backend.argsort(evals)
18     evals, evecs = evals[argsort], evecs[argsort]
19
20     # [Ne, Lz * Ly * Lx, Nc]
21     if apply_renorm_phase:
22         renorm_phase = backend.angle(evecs[:, 0])
23         evecs *= backend.exp(-1.0j * renorm_phase)[:, None]
24     return evecs.reshape(self.Ne, Lz, Ly, Lx, Nc), evals

```

stout CUDA 核主循环 (C2, stout\_smear.cu:160-199):

```

1  __global__ void stout_smear(complex<T> *U_out, const complex<T> *U_in, const T rho, const int Lx, const int Ly, const
    ↪ int Lz, const int Lt)
2  {
3      const int Nd = 4;
4      const int Nc = 3;
5      typedef Matrix<complex<T>, Nc> ColorMatrix;
6
7      const int volume = Lx * Ly * Lz * Lt;
8      const int x = blockIdx.x * blockDim.x + threadIdx.x;
9      const int mu = blockIdx.y * blockDim.y + threadIdx.y;
10     if (mu >= Nd - 1 || x >= volume) {
11         return;
12     }
13     const int X[Nd] = {Lx, Ly, Lz, Lt};
14     int coord[Nd] = {x % Lx, x / Lx % Ly, x / (Lx * Ly) % Lz, x / (Lx * Ly * Lz) % Lt};
15
16     ColorMatrix U(U_in + (mu * volume + x) * Nc * Nc);
17     ColorMatrix Q;
18     for (int nu = 0; nu < Nd - 1; ++nu) {
19         if (nu != mu) {
20             ColorMatrix staple1(U_in + (nu * volume + get_x(coord, X)) * Nc * Nc);
21             coord[nu] = (coord[nu] + 1 + X[nu]) % X[nu];
22             staple1 = staple1 * ColorMatrix(U_in + (mu * volume + get_x(coord, X)) * Nc * Nc);
23             coord[mu] = (coord[mu] + 1 + X[mu]) % X[mu];
24             coord[nu] = (coord[nu] - 1 + X[nu]) % X[nu];
25             staple1 = staple1 * adjoint(ColorMatrix(U_in + (nu * volume + get_x(coord, X)) * Nc * Nc));
26             coord[mu] = (coord[mu] - 1 + X[mu]) % X[mu];
27
28             coord[nu] = (coord[nu] - 1 + X[nu]) % X[nu];
29             ColorMatrix staple2(U_in + (nu * volume + get_x(coord, X)) * Nc * Nc);
30             staple2 = adjoint(staple2) * ColorMatrix(U_in + (mu * volume + get_x(coord, X)) * Nc * Nc);
31             coord[mu] = (coord[mu] + 1 + X[mu]) % X[mu];
32             staple2 = staple2 * ColorMatrix(U_in + (nu * volume + get_x(coord, X)) * Nc * Nc);
33             coord[nu] = (coord[nu] + 1 + X[nu]) % X[nu];
34             coord[mu] = (coord[mu] - 1 + X[mu]) % X[mu];
35
36             Q += staple1 + staple2;

```

```

37     }
38 }
39 Q = antiherm((Q * rho) * adjoint(U));

```

导数与动量相位 (C3, derivative.py:23-33 与 phase.py:41-53):

```

1 def derivative(n: int):
2     ret = []
3     num = -1
4     while n >= 0:
5         num += 1
6         n = n - pow(3, num)
7     n = n + pow(3, num)
8     for _ in range(num):
9         ret.append(n % 3)
10        n = n // 3
11    return tuple(ret[::-1])

```

```

1 def get(self, np: Tuple[int]):
2     npx, npy, npz = np
3     if np not in self.cache:
4         backend = get_backend()
5         self.cache[np] = backend.exp(npx * self.x + npy * self.y + npz * self.z)
6     return self.cache[np]
7
8 def get_cb2(self, np: Tuple[int]):
9     npx, npy, npz = np
10    if np not in self.cache_cb2:
11        backend = get_backend()
12        self.cache_cb2[np] = backend.exp(npx * self.x_cb2 + npy * self.y_cb2 + npz * self.z_cb2)
13    return self.cache_cb2[np]

```

小群投影 (C4 核心, gen\_hardcoded\_rep.py:226-245):

```

1 def reductionToLittleGroup(momentum, OhD_irep_name, parity, little_group_irep_name):
2     """
3     Reduce the OhD irrep to the little group irrep.
4     """
5     p = momentumSimplify(momentum)
6     matrix_little_group_irep = genLittleGroupIrrep(p, little_group_irep_name)
7     matrix_OhD_irep = genIrrepOhD(OhD_irep_name, parity)
8     ndim_little_group = matrix_little_group_irep["iden"].shape[0]
9     ndim_OhD = matrix_OhD_irep["iden"].shape[0]
10    reduction_matrix_colinear = np.full((ndim_little_group, ndim_OhD, ndim_OhD), S(0))
11    reduction_matrix = np.full((ndim_little_group, ndim_OhD), S(0))
12    for key in matrix_little_group_irep.keys():
13        reduction_matrix_colinear += np.einsum(
14            "ij,kk->kji",
15            np.array(matrix_OhD_irep[key]),
16            np.array(matrix_little_group_irep[key]),
17        )
18    for i in range(ndim_little_group):
19        reduction_matrix[i] = check_and_normalize_arrays(reduction_matrix_colinear[i])
20    return reduction_matrix

```

perambulator 反演 (C5 核心, perambulator.py:146-175):

```

1  def calc(self, t: int):
2      import cupy as cp
3
4      backend = get_backend()
5      from pyquda_utils.core import LatticeFermion, MultiLatticeFermion
6
7      if self.gauge_field_new:
8          self.dirac.loadGauge(self.gauge_field_smear) # loadGauge after
9          self.gauge_field_new = False
10
11      latt_info = self.latt_info
12      Lx, Ly, Lz, Lt = latt_info.size
13      Vol = Lx * Ly * Lz * Lt
14      Ne = self.usedNe
15      eigenvector_dagger = self._eigenvector_data_dagger
16      dirac = self.dirac
17      gx, gy, gz, gt = self.latt_info.grid_coord
18
19      SV = self._SV
20      VSV = self._VSV
21
22      from time import perf_counter
23
24      for eigen in range(Ne):
25          cp.cuda.runtime.deviceSynchronize()
26          s = perf_counter()
27          if self.MRHS:
28              print("Warning: use MRHS.")
29              V_MRHS = MultiLatticeFermion(latt_info, Ns)
30              for spin in range(Ns):

```

## 5 参考源清单

| 参考源                          | 类型 | 内容/作用                              |
|------------------------------|----|------------------------------------|
| quark_contract.py:110-213    | 仓库 | C1 主收缩: 味 $\rightarrow$ 传播子 + 邻接矩阵 |
| quark_contract.py:216-245    | 仓库 | C1 递归配对收缩 (符号/简并)                  |
| quark_diagram.py:15-79       | 仓库 | C1 邻接矩阵 $\rightarrow$ einsum 图分析   |
| quark_diagram.py:233-273     | 仓库 | C1 关联函数数值求值                        |
| quark_diagram.py:330-553     | 仓库 | C6 图化简 (连通分量拆分)                    |
| eigenvector.py:11-26         | 仓库 | C2 拉普拉斯算符                          |
| eigenvector.py:234-257       | 仓库 | C2 scipy/cupyx 求解                  |
| eigenvector.py:259-312       | 仓库 | C2 QUDA 求解                         |
| eigenvector.py:356-370       | 仓库 | C2 路径分发                            |
| stout_smear.cu:160-245       | 仓库 | C2 stout CUDA 核                    |
| elemental.py:279-338         | 仓库 | C3 协变导数 + 动量投影                     |
| derivative.py:23-33          | 仓库 | C3 导数 3 进制编码                       |
| phase.py:41-53               | 仓库 | C3 动量平面波相位                         |
| insertion/_init_.py:219-350  | 仓库 | C4 Insertion 算符 + 小群投影             |
| group_generator.py:25-73     | 仓库 | C4 $O_h^P$ 生成元                     |
| gen_hardcoded_rep.py:145-245 | 仓库 | C4 Wigner 转动 + 小群投影                |



| 参考源                            | 类型 | 内容/作用                    |
|--------------------------------|----|--------------------------|
| hardcoded_rep.py:10188         | 仓库 | C4 硬编码不可约表示表             |
| hadron_irrep.py:132-195        | 仓库 | C4 群元作用与投影               |
| perambulator.py:51-102         | 仓库 | C5 构造与参数                 |
| perambulator.py:146-209        | 仓库 | C5 反演 + 收缩               |
| backend.py:36-58               | 仓库 | PyQUDA 探测/版本约束           |
| test_quark_contract.py:12-222  | 仓库 | C1 符号测试 19 例（本机通过）       |
| test_simplify.py:72-367        | 仓库 | C6 化简测试 8 例（本机通过）        |
| test_eigenvector.py:25-59      | 仓库 | C2 参考比对测试（LFS，未运行）       |
| test_elemental.py:23-48        | 仓库 | C3 参考比对测试（LFS，未运行）       |
| test_perambulator.py:29-66     | 仓库 | C5 参考比对测试（需 QUDA，未运行）    |
| test_perambulator_mpi.py:10-82 | 仓库 | C5 MPI 片切分测试（需 QUDA，未运行） |
| README.md:3-18                 | 仓库 | 定位：四阶段一站式 + PyQuda 依赖    |
| TODO.md:3-13                   | 仓库 | 功能路线图                    |

## 6 结论

### 结论

**穷尽剖析结论：**核心部分候选 14 项（C1–C14），三态分配为 **5 深挖 / 8 浅析 / 1 排除**。最强竞争力是 **C1 自动 Wick 收缩引擎**：从强子味结构的符号代数出发，自动生成全部 Wick 配对、置换符号与传播子组合，并以邻接矩阵统一编码（介子/重子/反重子），再由 BFS 图分析自动翻译为 einsum 收缩表达式——这条“物理收缩 → 图 → 张量网”的自动化链是手写收缩脚本的朴素实现所不具备的。其次 **C4 群论自动算符构造**（小群投影 + Wigner 转动 + 硬编码不可约表示）与 **C2 三路径本征求解**（scipy/cupyx/QUDA + stout 三实现）构成完整的物理工作流支撑。

### 局限与风险

**未验证/局限：**全部 GPU/QUDA 数值路径（C2/C5）与数据相关测试（C3）本机未运行（缺 libquda.so 与 LFS 数据），竞争力中的性能数字无实测；排除项 C13/C14 为纯查表/工程配置；浅析项 C6–C12 未进入 15 步深挖，各自依赖的最深实现见参考源表；自动收缩的指数级分支在顶点数增大时的实际表现未评估。